

Relatório Técnico: Atividade 15

Título: Sumário do Resultado — Automação de Infraestrutura como Código (IaC) com Terraform **Autor:** Cleydson Soares de Freitas

Data: 10/06/2026

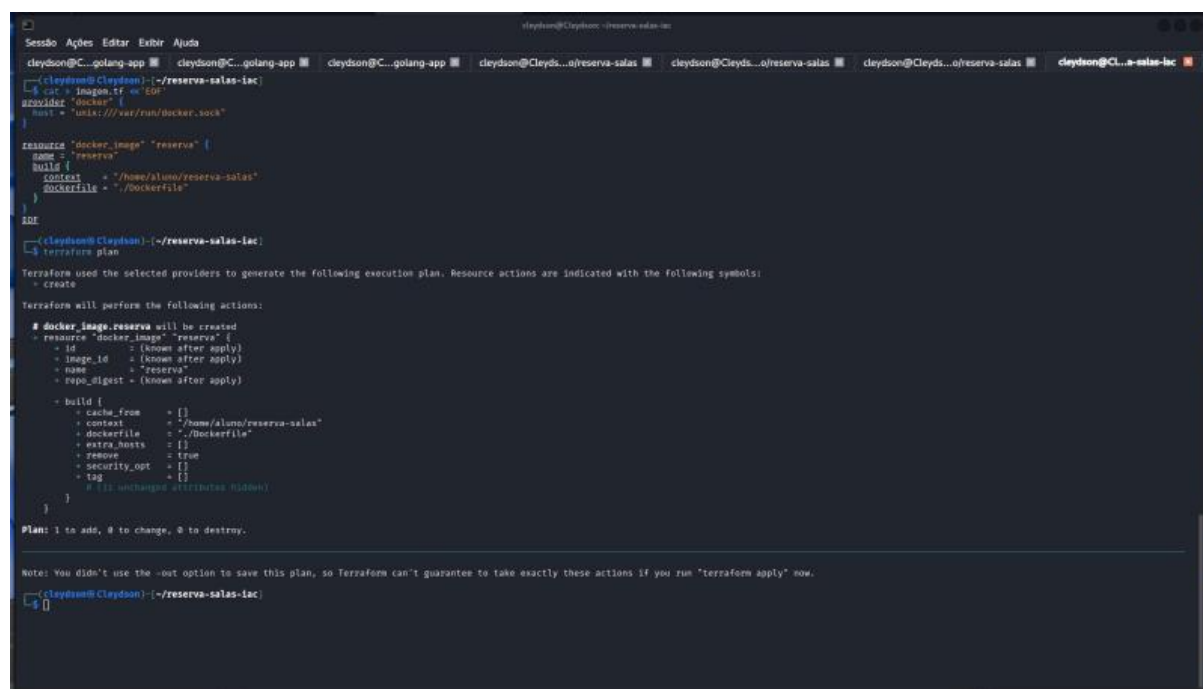
1. Objetivo

Migrar o gerenciamento da infraestrutura containerizada para o modelo declarativo de Infraestrutura como Código (IaC), utilizando o Terraform para automatizar todo o ciclo de vida dos recursos e ferramentas de monitoramento.

2. Desenvolvimento e Resultados

- **Construção Isolada:** Criação dos manifests providers.tf, imagem.tf e container.tf para gerenciar o build do Dockerfile e o deploy da aplicação de forma automatizada.
- **Resolução de Conflitos:** Superação de erros de porta duplicada (8080) limpando processos legados diretamente pelo terminal.
- **Expansão e Monitoramento:** Injeção das definições de containers do Prometheus e Grafana direto no código do Terraform.
- **Ciclo de Vida Limpo:** Execução do comando terraform destroy, comprovando a capacidade da ferramenta de rastrear o estado e remover 100% da arquitetura sem deixar resíduos no sistema de destino.

3. Evidências do Processo



```
Sessão Ações Editar Exibir Ajuda
cleydson@C...golang-app  cleydson@C...golang-app  cleydson@C...golang-app  cleydson@Cleyds...a/reserva-salas  cleydson@Cleyds...a/reserva-salas  cleydson@Cleyds...a/reserva-salas  cleydson@C...a-salas-lac

cleydson@Cleydson) ~/reserva-salas-lac
└─$ cat > imagem.tf < cat
provider "docker" {
  host = "unix:///var/run/docker.sock"
}

resource "docker_image" "reserva" {
  name = "reserva"
  build {
    context = "/home/Aluno/reserva-salas"
    dockerfile = "/Dockerfile"
  }
}

cleydson@Cleydson) ~/reserva-salas-lac
└─$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# docker_image.reserva will be created
+ resource "docker_image" "reserva" {
  + id          = (known after apply)
  + image_id    = (known after apply)
  + name        = "reserva"
  + repo_digest = (known after apply)

  + build {
    + cache_from = []
    + context    = "/home/Aluno/reserva-salas"
    + dockerfile = "/Dockerfile"
    + extra_hosts = []
    + remove     = true
    + security_opt = []
    + tag        = []
  }
}

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee it takes exactly these actions if you run "terraform apply" now.

cleydson@Cleydson) ~/reserva-salas-lac
└─$
```

```

(cleydson@cleydson)-[~/reserva-salas-iac]
$ terraform plan
docker_image.reserva: Refreshing state... [id=sha256:67529ee7a407fa8a664df3cc342433da2c1985fe82794a980e8c866efd822878reserva]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# docker_container.reserva will be created
+ resource "docker_container" "reserva" {
  + attach                = false
  + bridge                = (known after apply)
  + command               = (known after apply)
  + container_logs        = (known after apply)
  + container_read_refresh_timeout_milliseconds = 15000
  + endpoint              = (known after apply)
  + env                  = (known after apply)
  + exit_code             = (known after apply)
  + hostname              = (known after apply)
  + id                   = (known after apply)
  + image                 = "sha256:67529ee7a407fa8a664df3cc342433da2c1985fe82794a980e8c866efd822878"
  + init                 = (known after apply)
  + ipc_mode              = (known after apply)
  + log_driver            = (known after apply)
  + logs                  = false
  + must_run              = true
  + name                  = "reserva"
  + network_data          = (known after apply)
  + read_only             = false
  + remove_volumes       = true
  + restart               = "always"
  + rm                   = false
  + runtime               = (known after apply)
  + security_opts         = (known after apply)
  + shm_size              = (known after apply)
  + start                 = true
  + stdin_open            = false
  + stop_signal           = (known after apply)
  + stop_timeout          = (known after apply)
  + tty                   = false
  + wait                  = false
  + wait_timeout          = 60

  + healthcheck (known after apply)
}

```

```

+ healthcheck (known after apply)
+ labels (known after apply)

+ ports {
  + external = 8080
  + internal = 8080
  + ip       = "0.0.0.0"
  + protocol = "tcp"
}

+ volumes {
  + container_path = "/app/dados"
  + host_path      = "/home/aluno/reserva-salas/dados"
  + (3 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

```

```

(cleydson@cleydson)-[~/reserva-salas-iac]
$

```

```

(cleydson@cleydson)-[~/reserva-salas-iac]
$ terraform plan
docker_image.reserva: Refreshing state... [id=sha256:67529ee7a407fa8a664df3cc342433da2c1985fe82794a980e8c866efd822878reserva]
docker_container.reserva: Refreshing state... [id=33c33f0abd0f2e8986372a2fa563bd4cd699e0457036f74c397cc43cb2ded0f]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
/- destroy and then create replacement

Terraform will perform the following actions:

# docker_container.grafana will be created
+ resource "docker_container" "grafana" {
  + attach                = false
  + bridge                = (known after apply)
  + command               = (known after apply)
  + container_logs        = (known after apply)
  + container_read_refresh_timeout_milliseconds = 15000
  + endpoint              = (known after apply)
  + env                  = [
    + "GF_INSTALL_PLUGINS=grafana-clock-panel,grafana-simple-json-datasource",
    + "GF_SECURITY_ADMIN_PASSWORD=admin",
    + "GF_SECURITY_ADMIN_USER=admin",
    + "GF_USERS_ALLOW_SIGN_UP=false",
  ]
  + exit_code             = (known after apply)
  + hostname              = (known after apply)
  + id                   = (known after apply)
  + image                 = "grafana/grafana"
  + init                 = (known after apply)
  + ipc_mode              = (known after apply)
  + log_driver            = (known after apply)
  + logs                  = false
  + must_run              = true
  + name                  = "grafana"
  + network_data          = (known after apply)
  + read_only             = false
  + remove_volumes       = true
  + restart               = "always"
  + rm                   = false
  + runtime               = (known after apply)
  + security_opts         = (known after apply)
  + shm_size              = (known after apply)
  + start                 = true
  + stdin_open            = false
  + stop_signal           = (known after apply)
}

```

```
Sessão  Ações  Editar  Exibir  Ajuda
cleydson@C...golang-app  cleydson@C...golang-app  cleydson@Cleyds...o/reserva-salas  cleydson@Cleyds...o/reserva-salas  cleydson@Cleyds...o/reserva-salas  cleydson@CL...a-salas-lac

+ stop_timeout = (known after apply)
+ tty = false
+ user = "1001:1001"
+ wait = false
+ wait_timeout = 60

+ healthcheck (known after apply)

+ labels (known after apply)

+ ports {
  + external = 3000
  + internal = 3000
  + ip = "0.0.0.0"
  + protocol = "tcp"
}

+ volumes {
  + container_path = "/var/lib/grafana"
  + host_path = "/home/aluno/reserva-salas/grafana/data"
  # (2 unchanged attributes hidden)
}

# docker_container.prometheus will be created
+ resource "docker_container" "prometheus" {
  + attach = false
  + bridge = (known after apply)
  + command = (known after apply)
  + container_logs = (known after apply)
  + container_read_refresh_timeout_milliseconds = 15000
  + endpoint = (known after apply)
  + env = (known after apply)
  + exit_code = (known after apply)
  + hostname = (known after apply)
  + id = (known after apply)
  + image = "prom/prometheus"
  + init = (known after apply)
  + ipc_mode = (known after apply)
  + log_driver = (known after apply)
  + logs = false
  + must_run = true
  + name = "prometheus"
  + network_data = (known after apply)
  + read_only = false
  + remove_volumes = true
  + restart = "always"
  + ns = false
}
```

```
Sessão  Ações  Editar  Exibir  Ajuda
cleydson@C...golang-app  cleydson@C...golang-app  cleydson@C...golang-app  cleydson@Cleyds...o/reserva-salas  cleydson@Cleyds...o/reserva-salas  cleydson@Cleyds...o/reserva-salas  cleydson@CL...a-salas-lac

+ runtime = (known after apply)
+ security_opts = (known after apply)
+ size_size = (known after apply)
+ start = true
+ stdin_open = false
+ stop_signal = (known after apply)
+ stop_timeout = (known after apply)
+ tty = false
+ wait = false
+ wait_timeout = 60

+ healthcheck (known after apply)

+ labels (known after apply)

+ ports {
  + external = 9090
  + internal = 9090
  + ip = "0.0.0.0"
  + protocol = "tcp"
}

+ volumes {
  + container_path = "/etc/prometheus"
  + host_path = "/home/aluno/reserva-salas/prometheus"
  # (2 unchanged attributes hidden)
}

+ volumes {
  + container_path = "/prometheus"
  + host_path = "/home/aluno/reserva-salas/prometheus/data"
  # (2 unchanged attributes hidden)
}

# docker_container.reserva must be replaced
/- resource "docker_container" "reserva" {
  + bridge = (known after apply)
  + command = [
    "/reserva-salas",
  ] -> (known after apply)
  + container_logs = (known after apply)
  + cpu_shares = 0 -> null
  + dns = [] -> null
  + dns_opts = [] -> null
  + dns_search = [] -> null
  + endpoint = [] -> (known after apply)
  + env = [] -> (known after apply)
  + exit_code = (known after apply)
}
```

```

Sessão  Ações  Editar  Exibir  Ajuda
cleydson@C...golang-app  cleydson@C...golang-app  cleydson@C...golang-app  cleydson@Cleyps...o/reserva-salas  cleydson@Cleyps...o/reserva-salas  cleydson@Cleyps...o/reserva-salas  cleydson@CL...a-salas-lac

- group_add = [] → null
- hostname = "33c33f0abd0f" → (known after apply)
- id = "33c33f0abd0f2e986372a2fa563d4cd699e6457036f74c397cc43cbc2ded6f" → (known after apply)
- init = false → (known after apply)
- ipc_mode = "private" → (known after apply)
- log_driver = "json-file" → (known after apply)
- log_opts = {} → null
- max_retry_count = 0 → null
- memory = 0 → null
- memory_swap = 0 → null
- name = "reserva"
- network_data = [
  {
    gateway = "172.17.0.1"
    global_ipv6_prefix_length = 0
    ip_address = "172.17.0.3"
    ip_prefix_length = 10
    mac_address = "a8:97:a3:2d:b6:dd"
    network_name = "bridge"
    # (2 unchanged attributes hidden)
  }
] → (known after apply)
- network_mode = "bridge" → null # forces replacement
- privileged = false → null
- publish_all_ports = false → null
- runtime = "runc" → (known after apply)
- security_opts = [] → (known after apply)
- shm_size = 64 → (known after apply)
- stop_signal = (known after apply)
- stop_timeout = 0 → (known after apply)
- storage_opts = {} → null
- sysctls = {} → null
- tmpfs = {} → null
- working_dir = "/app" → null
# (19 unchanged attributes hidden)

- healthcheck (known after apply)

- labels (known after apply)

- volumes {
  container_path = "/app/dados" → null
  host_path = "/home/aluno/reserva-salas/dados" → null
  read_only = false → null
  # (2 unchanged attributes hidden)
}
+ volumes {
  container_path = "/app/dados"
}
# (1 unchanged block hidden)

Plan: 3 to add, 0 to change, 1 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
(cleydson@Cleydson) ~/reserva-salas-lac
```

```

- container_path = "/app/dados" → null
- host_path = "/home/aluno/reserva-salas/dados" → null
- read_only = false → null
# (2 unchanged attributes hidden)
}
+ volumes {
  container_path = "/app/dados"
  host_path = "/home/aluno/reserva-salas/dados"
# (2 unchanged attributes hidden)
}
# (1 unchanged block hidden)
}

Plan: 3 to add, 0 to change, 1 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
(cleydson@Cleydson) ~/reserva-salas-lac
```

```

Sessão  Ações  Editar  Exibir  Ajuda
cleydson@C...golang-app  cleydson@C...golang-app  cleydson@C...golang-app  cleydson@Cleyps...o/reserva-salas  cleydson@Cleyps...o/reserva-salas  cleydson@Cleyps...o/reserva-salas  cleydson@CL...a-salas-lac

# docker_image.reserva will be destroyed
resource "docker_image" "reserva" {
  id = "sha256:67529ee7a407f8a8a64df3cc342433da2c1985fe82794a980e8c866efd822878reserva" → null
  image_id = "sha256:67529ee7a407f8a8a64df3cc342433da2c1985fe82794a980e8c866efd822878" → null
  name = "reserva" → null
# (1 unchanged attribute hidden)

- build {
  build_arg = {} → null
  build_args = {} → null
  cache_from = [] → null
  context = "." → null
  cpu_period = 0 → null
  cpu_quota = 0 → null
  cpu_shares = 0 → null
  dockerfile = "/Dockerfile" → null
  extra_hosts = [] → null
  force_remove = false → null
  label = {} → null
  labels = {} → null
  memory = 0 → null
  memory_swap = 0 → null
  no_cache = false → null
  pull_parent = false → null
  remove = true → null
  security_opt = {} → null
  shm_size = 0 → null
  squash = false → null
  suppress_output = false → null
  tag = {} → null
# (11 unchanged attributes hidden)
}

Plan: 0 to add, 0 to change, 4 to destroy.
docker_container.grafana: Destroying... [id=8fad88e99b3feddc79051714e95d09cfb1e7c59ff837bd5715081aa1cc9e9df]
docker_container.reserva: Destroying... [id=f7f062a6dd2c9e570d012487ee7998cd0546207fc9343864bd64de97df254936]
docker_image.reserva: Destruction complete after 1s
docker_image.reserva: Destruction complete after 0s
docker_container.grafana: Destruction complete after 1s
docker_container.prometheus: Destroying... [id=8ca266899dc983dd157a09359a5ac0e76565430b831eeef1378a6d1b88a220ea]
docker_container.prometheus: Destruction complete after 0s

Destroy complete! Resources: 4 destroyed.
(cleydson@Cleydson) ~/reserva-salas-lac
```

4. Conclusão

A adoção de IaC com Terraform provou que a infraestrutura pode e deve ser tratada como software. Esse modelo elimina erros manuais, garante que ambientes idênticos possam ser recriados rapidamente e viabiliza a escalabilidade segura necessária para DevSecOps.